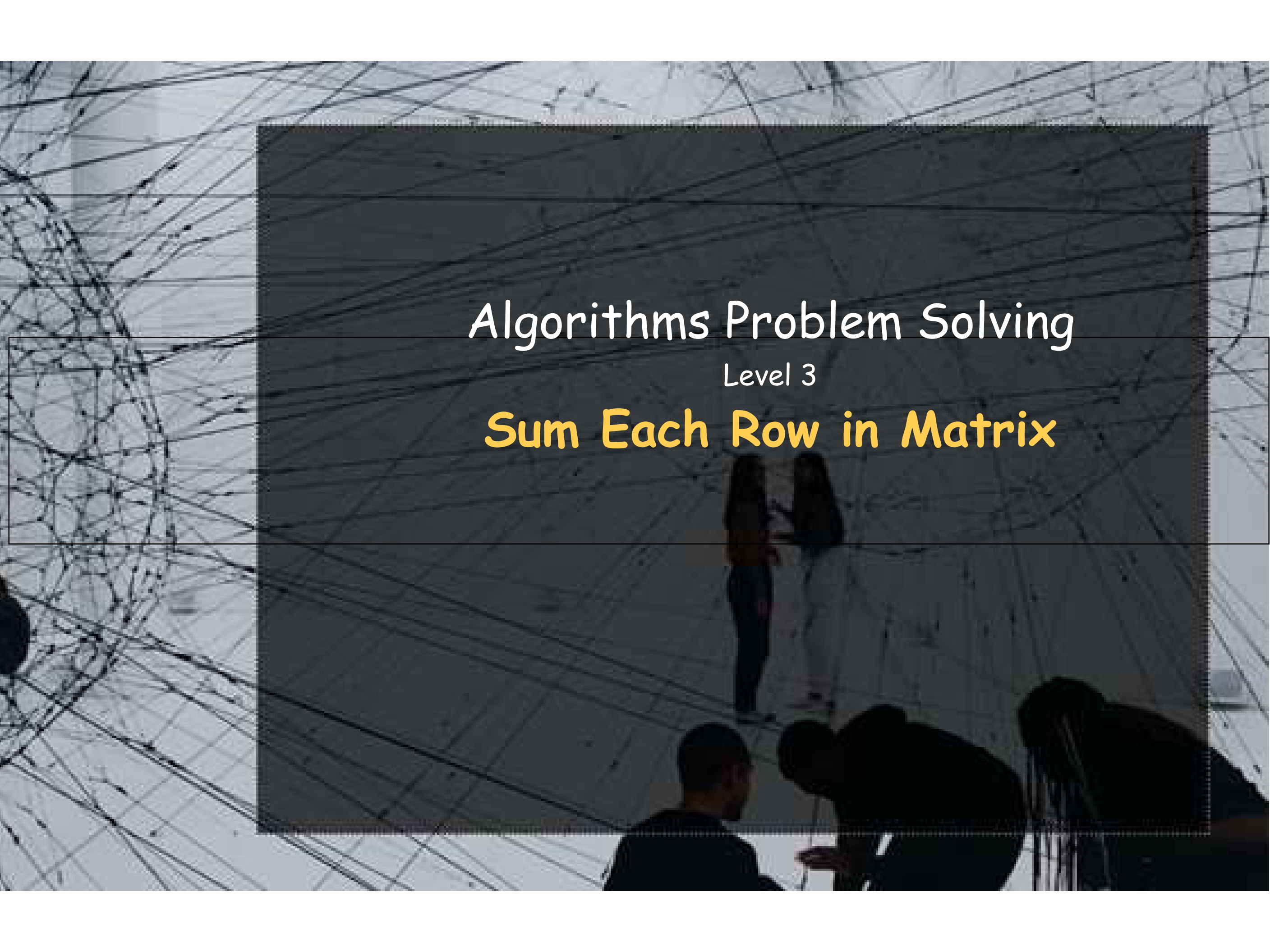


Algorithms Problem Solving

Level 3

Sum Each Row in Matrix



Algorithms Problem Solving

Level 3

Sum Each Row in Matrix

Definition

Sum Each Row in Matrix

-  The program creates a 3x3 matrix containing random numbers between 1 and 100.
-  It then calculates and prints the sum of each row separately.

Explanation

2D Matrix

-  A matrix is a two-dimensional data structure consisting of rows and columns.

-  We use ``vector<vector<int>>`` to store the matrix.

Filling the Matrix

-  The matrix is filled with random numbers using ``rand()`.`

Calculating Row Sums

-  We calculate the sum of the elements in each row separately.

-  A ``rowSum`` variable is reset to ``0`` for every new row.

Nested ``for`` Loops

-  The outer loop moves through the rows.

-  The inner loop moves through the columns of the current row.

Procedures

1 Create a 3x3 matrix using:

```
vector<vector<int>> matrix(3, vector<int>(3));
```

2 Fill the matrix with random numbers from 1 to 100 using:

```
FillMatrixWithRandomNumber(matrix, 3, 3);
```

3 Print the original matrix using:

```
PrintMatrix(matrix, 3, 3);
```

4 Calculate and print the sum of each row using:

```
PrintRowSums(matrix, 3, 3);
```

 `FillMatrixWithRandomNumber()`

 Fills the matrix with random numbers using:

```
rand() % 100 + 1
```

 `PrintMatrix()`

 Prints the matrix in a formatted way.

 `setw()` can be used to align the columns.

 `PrintRowSums()`

 Calculates the sum of each row and prints it in the following format:

```
Row 1 Sum = ...
```

```
Row 2 Sum = ...
```

```
Row 3 Sum = ...
```

💡 Example

The following is a 3x3 random matrix:

53 43 6

65 83 48

64 30 36

The following are the sum of each row in the matrix:

Row 1 Sum = 102

Row 2 Sum = 196

Row 3 Sum = 130

➡ Result

Row 1: $53 + 43 + 6 = 102$

Row 2: $65 + 83 + 48 = 196$

Row 3: $64 + 30 + 36 = 130$

Note

- ◆ For calculating the sum of each row, `rowSum` should be declared or reset inside the outer loop.
- ◆ If `rowSum` is kept outside the outer loop without resetting it, the values from previous rows will be included in later sums.

⊘ Warning

⊘ Warning

- ⚡ Do not confuse the sum of each row with the sum of the entire matrix.
- ⚡ If `rowSum` is not reset for every row, the results will be cumulative instead of separate row sums.



Comparison

▨ Sum of Each Row

- ✓ Uses `rowSum` for each individual row.
- ✓ `rowSum` is reset when a new row starts.
- ✓ Produces a separate result for every row.
- ✓ Example: `102`, `196`, `130`.

▩ Sum of Entire Matrix

- ✓ Uses one total sum for all elements.
- ✓ The sum is not reset between rows.
- ✓ Produces one final result for the entire matrix.
- ✓ Example: $102 + 196 + 130 = 428$.

✦ Summary

- ✓ Use nested `for` loops to process a 2D matrix.
- ✓ The outer loop processes each row.
- ✓ The inner loop processes the elements inside the current row.
- ✓ Use `rowSum` to calculate the sum of each row.
- ✓ Reset `rowSum` for every new row.
- ✓ `rand() % 100 + 1` generates values from 1 to 100.

Thank you 😊